

# AppleYOLO: Apple yield estimation method using improved YOLOv8 based on Deep OC-SORT

Shiting Tan, Zhufang Kuang\* and Boyu Jin

College of Computer and Mathematics, Central South University of Forestry and Technology, Changsha 41004, Hunan, China

## ARTICLE INFO

### Keywords:

Apple yield estimates  
FasterNet  
Focal Modulation  
KernelWarehouse  
Deep OC-SORT

## ABSTRACT

The accuracy of apple yield estimates is critical to orchard management. Existing apple yield estimation methods still lack in accuracy and efficiency. To solve this challenge, an Apple yield estimate method based on YOLOv8 and Deep OC-SORT (AppleYOLO) is proposed in this paper. To adequately learn the edge information of the apples, the lightweight FasterNet is used as the backbone portion of the AppleYOLO. In order to make AppleYOLO accurately capture the contextual information and improve the spatial perception of the apples, the Focal Modulation is designed after its backbone. To address the ability to extract complex features, the dynamic convolutional KernelWarehouse with efficient parameters is employed in the feature fusion part of AppleYOLO. For overcoming the problem of unstable tracking and repeated counting when detecting apples in real time, the Deep OC-SORT is deployed in AppleYOLO. In our customised dataset, validated by ablation experiments, the introduction of FasterNet, Focal Modulation, and KernelWarehouse enhances the detection performance of AppleYOLO. In comparison with the benchmark model YOLOv8, AppleYOLO achieves mAP50 and mAP50-95 of 98.5% and 79.8%, respectively, with 1% and 5.1% improvement. Moreover, the performance of AppleYOLO is superior compared to other state-of-the-art methods, such as YOLOv9-c, RT-DETR, YOLOv8 and YOLOv7. The experiments prove that AppleYOLO is able to achieve the goal of high accurate and high efficient.

## 1. Introduction

Agriculture is vital to a nation's very foundation, and the high accurate and efficient agricultural crop yield data is essential for enabling precise agricultural management, ensuring national food security, and accurately maintaining a strategic advantage in global food trade (Huang et al., 2023). In this context, it is particularly crucial to undertake yield estimation work for orchard fruits. Numerous studies are focused on enhancing the efficiency of fruit yield estimation and addressing the challenges faced during the yield estimation process. An integrated YOLOv4-tiny method with Channel and Spatial Reliability Discriminative Correlation Filter (CSR-DCF) algorithm is proposed to address the misidentification issue caused by a large number of fruits with highly similar appearances, thereby improving the accuracy of apple yield counting (Gao et al., 2022). An improved tomato clusters counting method combining the YOLOv5-4D and the ByteTrack is introduced by Rong et al. (2023), effectively overcoming challenges in automatic yield estimation in smart greenhouse automation. In Fu (2021), a computational complexity optimized YOLOv3-tiny network is proposed to automatically detect the kiwifruit in orchards. A new method combining the CenterNet model and a patch matching model based on the Kuhn-Munkres algorithm is introduced, successfully solving the issue of duplicate counting in sequential images of apples and oranges when performing fruit yield estimation (Xia et al., 2022). In Oliveira et al. (2023), a method combining the Linear Regression and the Support Vector Machine algorithm is pro-

posed to address efficiency and accuracy issues in strawberry yield estimation. In Bai et al. (2019), a method that integrates the Leaf Area Index (LAI) from Landsat satellite data and the WOFOST model is introduced to enhance the accuracy of yield estimation for perennial jujube crops on a field scale, effectively addressing the accuracy issues of traditional methods. A deep learning approach based on the YOLO-V4 model is proposed by Mirhaji et al. (2021) for automatically detecting and counting oranges in orchards, addressing the fruit load estimation issue. A deep learning algorithm combining the OrangeYolo and the OrangeSort is proposed, effectively solving the accuracy and duplicate counting problems in computer vision-based citrus counting (Zhang et al., 2022).

Existing research on fruit yield estimation methods has not yet concurrently addressed accuracy, real-time performance, and computational complexity, and cannot effectively meet the both high accurate and high efficient requirements. Therefore, an Apple yield estimation method in the complex orchard environment using improved YOLOv8 based on Deep OC-SORT (AppleYOLO) is proposed. Meanwhile, the computational complexity of the model is reduced as much as possible to accommodate more hardware devices. As shown in Table 1, comparison to existing works, we optimize both the backbone and neck portions of the AppleYOLO and introduce the state-of-the-art target tracking technique. By applying AppleYOLO, it is possible to accurately locate the apple on the tree and quickly count the number of apples with error-free detection, thus improving the accuracy of apple yield estimation in the orchard. The main contributions of this paper are summarized as follows:

1. An Apple yield estimate method based on YOLOv8 and Deep OC-SORT (AppleYOLO) is proposed to ad-

\*Corresponding author

✉ zfkuan@163.com (Z. Kuang)

ORCID(s): 0000-0002-1445-3217 (Z. Kuang)

**Table 1**  
Comparison of the existing works.

| Reference           | Application target | Backbone | end of the Backbone | Neck | Tracker | Accuracy | Computational complexity |
|---------------------|--------------------|----------|---------------------|------|---------|----------|--------------------------|
| Xiong et al.,(2023) | Litchi             |          |                     | ✓    |         | ✓        |                          |
| Zhang et al.,(2023) | Apple leaf         | ✓        |                     | ✓    |         | ✓        |                          |
| Li et al.,(2024b)   | Wheat ears         | ✓        |                     | ✓    | ✓       | ✓        | ✓                        |
| Du et al.,(2023)    | Strawberry         |          | ✓                   | ✓    |         | ✓        | ✓                        |
| Li et al.,(2023)    | Tea Buds           |          |                     |      | ✓       | ✓        |                          |
| Our paper           | Apple              | ✓        | ✓                   | ✓    | ✓       | ✓        | ✓                        |

dress the challenges of apple yield estimation in large orchards. Figure 1 displays the four key components of the AppleYOLO network architecture: the Backbone network, the Neck network, the Head network, and the Tracker section.

2. To improve the network's ability to learn the edge information of the input image, such as the contour and texture of apples, the FasterNet is introduced in the backbone network of AppleYOLO. It enhances the efficiency of learning the edge information of objects without increasing the memory access time of the network model.
3. For extracting apple size information, locating the position of apples, and improving the spatial recognition of objects, the Focus Modulation (FM) mechanism is used to improve the interpretability of AppleYOLO and its ability to capture key information.
4. The complex feature extraction ability of AppleYOLO Neck network is improved by using KernelWarehouse (KWConv) in order to reduce the occlusion and interference caused by tree trunks, branches and leaves on apple detection.
5. The tracking algorithm Deep OC-SORT is utilised as the Tracker section of AppleYOLO to solve the problem of unstable tracking and repeated counting in real-time apple detection. It ensures that each detected apple is assigned a unique identification number, thereby increasing the accuracy of apple yield estimation.

## 2. Related work

The efficient apple yield estimation methods can significantly enhance orchard management efficiency and promote precision in agricultural management. In contrast, the traditional orchard fruit yield estimation methods usually rely on the judgment of experienced farmers or sample counting in specific areas to extrapolate the expected total yield of the orchard. In Wulfsohn et al.(2012), a three-stage systematic sampling procedure is employed to unbiasedly estimate the number of fruits in orchards, addressing the challenge

of accurate estimation due to significant fluctuations in tree for orchard yield forecasting. However, in Gómez-Lagos et al.(2023), a fruit yield prediction method based on spectral vegetation data and machine learning techniques (SBM-Fruit) is used to address inefficiencies of resource allocation and logistics planning in the fruit supply chain caused by inaccurate yield estimates. The study points out that the traditional random sampling methods generally have a prediction error of about 10% when estimating fruit yield, but these can be as high as 30% in actual field operations, leading to increased resource costs, fruit losses, packaging waste, and decreased fruit qualities. In the review (He et al.,2022), vegetation index information is used for yield prediction, and mainly focuses on the overall health and growth status of vegetation. However, it ignores attention to specific details. At the same time, the literature also emphasizes the rapid development of computer vision technology based on image processing and deep learning in the agricultural field. Computer vision technology excels in detailed crop analysis and counting, particularly in identifying and counting individual fruits with high accuracy (Xiong et al.,2023). For instance, the YOLOv5s detection model is used to count grape clusters under field conditions and achieves achieving an average counting accuracy of 84.9% (Shen et al.,(2023). Building on the work of predecessors, our study uses the most advanced computer vision technology to address the three major challenges faced by existing orchard yield estimation models: insufficient identification accuracy and low precision, inability to meet the needs of real-time estimation, and models are too large or complex to run on small devices.

To construct an efficient method for estimating apple yield, it is typically necessary innovate in apple detection models and counting techniques. Existing studies are dedicated to improving the performance of object detection models by focusing on the backbone or neck sections of the network. In terms of counting methods, it is common to integrate target tracking algorithms with object detection models. For object detection models, it is essential to accurately identify the edges of objects to distinguish different entities.

Many researchers are committed to extracting edge information from objects and effectively building the backbone network. For example, in [Redmon and Farhadi\(2018\)](#), a deep neural network known as DarkNet53 is utilized as the backbone of the object detection model to accurately extract fundamental image features, such as edges, textures, and shapes. An improved cross-stage partial-connection Darknet53 is used as the backbone to address the problem of image information loss when precisely detecting railway objects in complex railway scenes ([Ye et al.,2022](#)). Although its detection accuracy reaches up to 94%, the model size is too large to deploy on embedded devices. An EfficientViT high-speed vision converter that can be used as a backbone part of the model is proposed to solve the problem that traditional converter models are not applicable in real-time applications due to high computational cost ([Liu et al.,\(2023\)](#)). Meanwhile, it also achieves a good balance between speed and accuracy. A method of using EfficientViT as the backbone is proposed for solving the problem of detecting small boat targets in synthetic aperture radar (SAR) imagery amidst complex backgrounds and noise interference ([Feng et al.,2023](#)). Although the model satisfies the requirements for real-time performance and small parameter size, there is still space for improvement in detection accuracy. A novel visual Transformer named Swin Transformer is employed to enhance efficiency and accuracy in visual tasks ([Liu et al.,2021](#)). It also addresses the challenges of high-resolution images and scale variation, and can serve as a universal backbone for computer vision. A Swin Transformer based model is used to address the problem of surface defect detection in manufacturing industry [Üzen et al.\(2022\)](#). it also harnesses Swin-transformer module to obtain global semantic features from initial features and achieves pixel-level accurate recognition through multi-feature integrated network. Its experimental results show that the model outperforms existing state-of-the-art models in the detection problem, but the computer requirements are too high to be supported by small hardware devices.

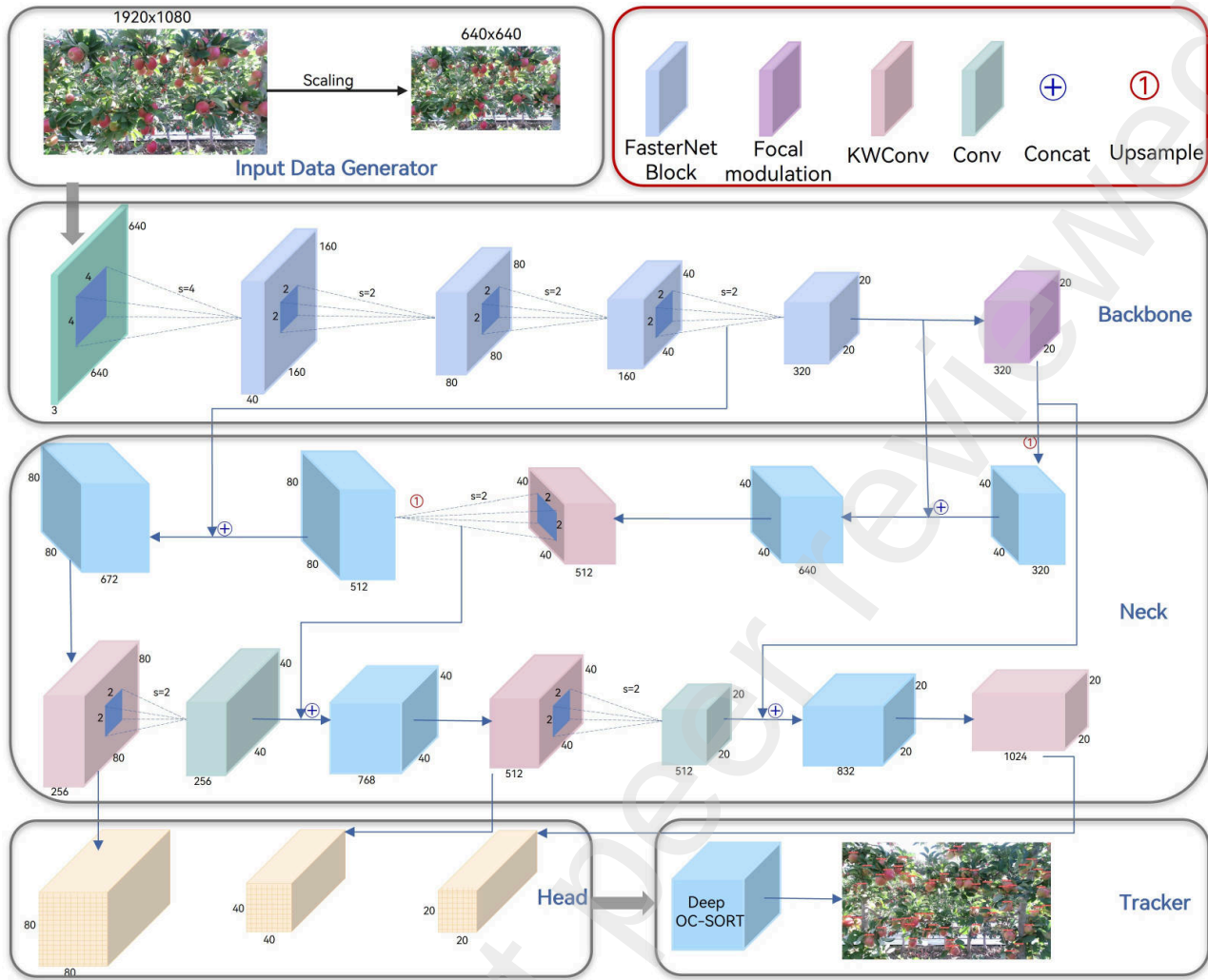
To deal with the long-distance dependency problem, some studies make the model capture richer contextual information by adding a pooling layer or other same function mechanisms after the backbone network to improve the recognition of object size and position. A method known as Fast Spatial Pyramid Pooling (SPPF) is deployed behind a backbone network to enhance the capability of sensory field and feature expression for Fusarium head blight spike detection in wheat ([Gao et al.,2024](#)). A Large Separable Kernel Attention (LSKA) is proposed to address the problem of high computational cost due to large convolutional kernels in visual attention networks ([Lau et al.,\(2024\)](#)). In addition, it maintains the ability of capturing the long distance dependency of the images to improve the accuracy of recognizing and segmenting the images. In the Real-Time Detection TRansformer (RT-DETR) method, an Attention-based Intra-scale Feature Interaction (AIFI) mechanism is used for solving the problem of accuracy and computational efficiency in object detection and recognition ([Lv et al.,2023](#)). And it en-

hances the ability to handle advanced features in RT-DETR method.

Many researchers optimize the feature fusion network part (Neck) of the detection model to make the model more accurately capture multi-scale features such as the size and shape of objects in the images. For instance, a method called Bidirectional Transposed Feature Pyramid Network (BTFPN) is employed to address the issue of apple leaf disease feature information loss during feature fusion and communication ([Zhang et al.,2023](#)). Although the recognition accuracy of the method in apple leaf disease detection is higher than other methods, its number of parameters and calculations reaches 49.13M and 114.8G, respectively, so that it cannot meet the requirements of model lightweight and high efficiency. A method that replaces the PANet structure of YOLOv7 with BIFPN is proposed to achieve more efficient fusion of wheat ears features ([Li et al.,2024b](#)). However, in the case of severe occlusion, there is still a room for improving the optimization for the detection of wheat ears. An approach which introduces the DCNv3 into the YOLOv7 model is proposed for improving the feature extraction capability for non-regular strawberries ([Du et al.,2023](#)). While the integration of DCNv3 really improve the method's ability of extracting important features, the more parameters that it introduces are not capable of fulfilling the requirements of high efficiency.

For the counting methods on the tracking side, the purpose is to achieve dynamic tracking of targets in consecutive frames within a video ([Ren et al.,2021](#)). An effective tracking algorithm maintains stable tracking of the targets and minimizes false tracking and tracking loss under complex conditions. A model fusing the YOLOv5x and the DeepSort algorithms is proposed to address the mutual occlusion problem caused by different movement speeds among individuals during sheep counting ([Cao et al.,2023](#)). A tea bud counting method based on the YOLOv5 and the Kalman filter tracking algorithm is proposed for estimating tea yield and its counting results are highly correlated with manual counting results ([Li et al.,2023](#)). In [Cui et al.\(2023\)](#), a method for rice field seedling drop counting based on improved the YOLOv5s and the ByteTrack tracking algorithm is used to accurately estimate the seedling deficiency rate during rice cultivation. Moreover, the counting accuracy of the method reaches up to 93.2%. [Maggiolino et al.\(2023\)](#) proposes an algorithm called Deep OC-SORT, which innovatively and adaptively integrates object appearance features into motion-based multi-target tracking methods to improve the accuracy and robustness of tracking. The results of their experiments indicate that the Deep OC-SORT is superior to DeepSort, ByteTrack and StrongSORT++ in terms of various performance metrics.

However, the existing works have not a more efficient method for apple yield estimations. Therefore, an apple yield estimations method named AppleYOLO is proposed , which is accurate, efficient, and adaptable to multiple hardware devices. The structure of the article is as follows: Section 3 introduces the dataset used in the research and



**Figure 1:** The detailed network architecture of AppleYOLO.

the design of the apple yield estimation method; Section 4 presents the experimental results of the apple yield estimation method; and Section 5 articulates the conclusions reached by this research.

### 3. Materials and methods

The construction of a sample dataset of images of apples on the trees is a key to realize accurate and efficient apple yield estimations. Therefore, it is necessary to obtain the images of apples on the trees in a real situation. Through exchanges with other academics, 797 high-definition apple images in real orchards are successfully obtained. Before constructing the apple dataset, 797 apple images are randomly divided into a training set of 637 and a validation set of 160 according to 8:2.

However, the small number of original apple images may lead to the model overfitting during the training process, which cannot effectively learn the location of apple growth in real scenes. In this case, the training performance of the model may show large fluctuations and lack of stability, lead-

ing to poor performance on data that has not been seen before. Therefore, a total of 1911 training images are obtained by data augmentation of the training set using Motion blur and adding Gaussian noise methods (Figure 2). The data processing methods are taken to increase the diversity of the data and the difficulty of model training to avoid the above as much as possible. Meanwhile, to further prevent the model from overfitting, the last 50 epochs early stopping method is used to set up the experiment. This approach is intended to monitor the model's performance on the validation set and the model will stop training when the performance no longer improves.

Then, following the guidance of the agriculturists, the exact location of apples in an image are determined and a single-row annotation method is employed for all images. As shown in Figure 3 (a), the camera photographs individual fruit trees in rows to obtain a front view of individual fruit trees. Similarly, the fruits in individual apple trees are annotated on a row-by-row basis (Figure 3 b). The aim of this is to avoid the model recognizing apples in the back row of an image, which in turn improves the accuracy of yield pre-



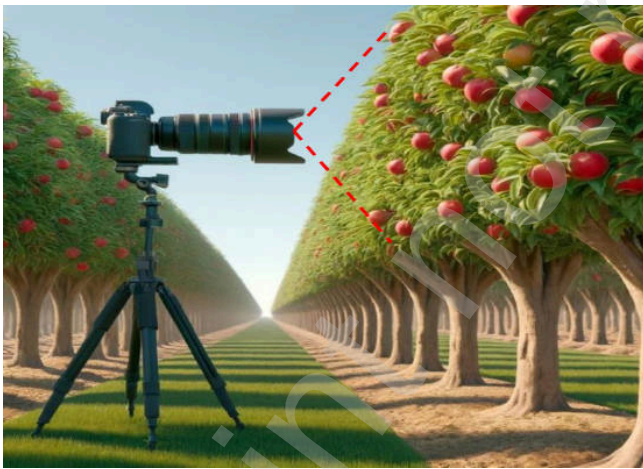
(a) Original image in the orchard



(b) Motion blur image



(c) Gaussian noise image

**Figure 2:** Examples of original and augmented images.

(a) Taking front view images of apple trees



(b) Labeling of front row fruits

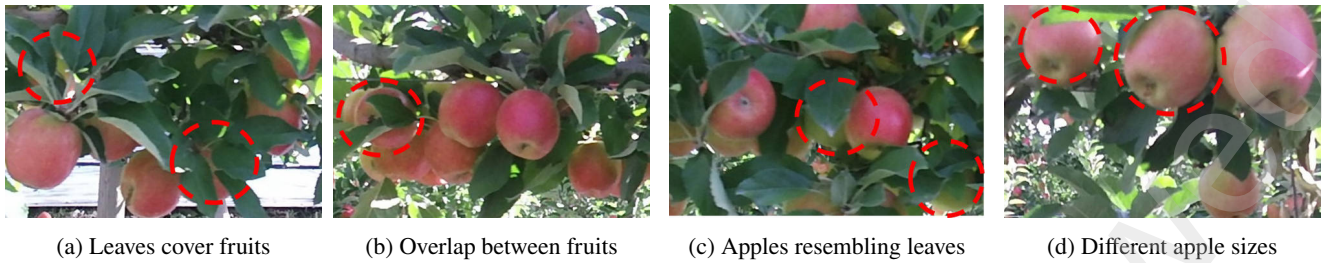
**Figure 3:** single-row annotation method.

diction. With the dataset prepared and the means of annotation determined, 2071 images are correctly annotated using the Labeling image annotation software to obtain an apple dataset that can be used for model training.

### 3.1. AppleYOLO

We find through field surveys with agriculturists that apple leaves cover the fruits more seriously (Figure 4 a), and overlapping between fruits (Figure 4 b). And besides the

red fruits, green apples that resemble the leaves also appear frequently (Figure 4 c) and the sizes of the fruits vary (Figure 4 d). This creates challenges in identifying to apples with an error-free margin and realizing high-precision apple yield estimates. For large orchards, it is necessary not only to have high-precision recognition technology, but also to optimize the performance of the model. The aim is to ensure real-time identification and yield estimation, which in



**Figure 4:** The factors affecting the accuracy of apple yield estimates.

turn enables efficient management and operation of the orchard. The AppleYOLO, which is well balanced between accurate and efficient and it can effectively solve the above problems.

Therefore, the state-of-the-art YOLOv8 network is redesigned to build the AppleYOLO for use in real-world scenarios. Different from any other network model, AppleYOLO employs a faster backbone network called FasterNet (Chen et al., 2023), which is capable of reducing redundant computations and memory accesses as well as efficiently extracting features of apple images. After the backbone network, a more powerful feature aggregation method called Focal Modulation (FM) mechanism is employed to enhance the model's interpretability of apple images and the ability of capturing key information (Yang et al., 2022). In the feature fusion section of AppleYOLO, the Towards Parameter Efficient Dynamic Convolutional KernelWarehouse (KWConv) is used to improve the parametric efficiency of the model while enhancing its feature characterization (Li and Yao, 2023). While the strategy ensures that the model is efficient, it also optimizes the processing and recognition performance for complex features. Moreover, the Deep OC-SORT tracking algorithm is integrated after the detection head to accurately track apple objects in moving videos, achieving error-free counting. Figure 1 illustrates the detailed network architecture of AppleYOLO.

- (1) In Backbone, for an input image with a channel number of 3 and a size of 640x640, the FasterNet's FasterNetBlock efficiently extracts the shallow features in the image, such as apple colors, contours, and textures. And the Focal Modulation mechanism aggregates multi-scale features to provide more comprehensive contextual information for the subsequent detection layer.
- (2) In Neck, it concatenates the shallow features obtain by Backbone with deeper semantic features. As can be seen from Figure 1, the resolution of the feature map is increased after the upsampling layer. Then, the feature maps from Backbone are merged with the upsampled feature maps using the Concat operation, which ensures that features at different scales are effectively fused. The KWConv is used to dynamically adjust the convolution kernel to optimize the characterization of features. And several regular convolutions (Conv) are responsible for generalized processing of features, including edge detection, texture recognition, and more. The combina-

tion of KWConv and Conv allows the model to capture complex, high-level features during the feature fusion process as well as maintain effective handling of the underlying features.

- (3) In Head, for increasing the model's focus and accuracy, a decoupled head idea is applied in the detection head part, which means that the classification and localization tasks are handled separately. And there are separate detectors for each scale. The detector consists of a set of convolutional layers and fully connected layers that are used to predict the bounding box at each scale. The design allows AppleYOLO is able to more accurately capture the features of the apple at all scales, thereby improving the overall detection accuracy.
- (4) In Tracker, the object detection part is responsible for detecting apples in an image in real time and recognizing the position and size of each apple. Subsequently, the results of these detections are passed to the Deep OC-SORT algorithm. The Deep OC-SORT algorithm utilizes this information to track the movement of each apple between video frames and thus achieve a continuous count of apples. The interaction ensures accurate counting of apples in dynamic environments and maintains a high level of accuracy even when the apples are moved or obscured from each other.

### 3.2. Apple detection model

#### 3.2.1. FasterNet

Some studies focus on reducing the number of floating point operations (FLOPs), but actually it is not always possible to reduce the latency. It is mainly caused by the low efficiency of FLOPs. Furthermore, the feature maps between most of the different channels are highly similar and redundancy of feature maps occurs, that lead to the existence of redundant computations. However, FasterNet revisits fast neural networks in terms of the number of memory interviews and redundant computations of the operators by using a novel partial convolution (PConv) to exploit the redundancy of the feature maps. The intention is to build fast neural networks in the truest sense of the word.

Figure 5 (b) demonstrates the working principle of PConv. As distinct from regular convolution, only a part of the channels of PConv are involved in the convolution operation for spatial feature extraction, and the rest of the channels remain unchanged. At this point the number of memory in-

interviews for PConv is

$$h \times w \times 2c_p + k^2 \times c_p^2 \approx h \times w \times 2c_p \quad (1)$$

assuming that the ratio of channels involved in the convolution operation to the original channels is  $r$ , when  $r = \frac{c_p}{c} = \frac{1}{4}$ , the number of memory interviews of PConv is  $\frac{1}{4}$  of that of the regular Conv. Concurrently, the FLOPs of PConv is

$$h \times w \times k^2 \times c_p^2 \quad (2)$$

when  $r = \frac{1}{4}$ , PConv is only  $\frac{1}{16}$  of the conventional convolution. As a result, PConv reduces both the number of memory interviews and redundant computations. In brief, the PConv reduces the computational burden while maintaining the focus on the key features. The selective convolutional approach allows the network to focus more on the important features, thereby improving the efficiency and accuracy of feature extraction.

Figure 5 depicts the overall architecture of FasterNet, which is a fast neural network build on the basis of PConv. And it can be used as a generalized backbone network to extract basic image features from the input images like the ResNet and the DarkNet. In addition, it can also obtain higher running speed than other networks on various devices without affecting the accuracy of various visual tasks. The FasterNet is composed of four layered stages, each of which is followed by an embedding layer (regular convolution  $4 \times 4$  with a step size of 4) or a merging layer (regular convolution  $2 \times 2$  with a step size of 2) for spatial downsampling and channel count expansion. Assuming that the height and width of the input images are both 640, according to the feature map calculation formula (3), the feature maps after the four stages are  $160 \times 160$ ,  $80 \times 80$ ,  $40 \times 40$ , and  $20 \times 20$ , respectively. The FasterNet Block is stacked at each stage of the FasterNet, and each FasterNet Block has a PConv layer. Besides, 2 point-wise convolutions (PWConv) are attached after the PConv. The combination of PConv and PWConv forms an inverted residual structure (Sandler et al., 2018), which is an effective network design for increasing network depth while maintaining or reducing computational effort. In addition to the above operations, the Batch Normalization and the Activation Layers are indispensable for high-performance neural networks. However, a study shows that overuse of these layers throughout the network, which may limit the feature diversity and thus impair the performance (Chen and Ho, 2021). Therefore, the FasterNet only puts the Batch Normalization and the Activation Layers in the middle of the 2 PWConv. The size of the output feature map is

$$f_{out} = (f_{in} - k)/s + 1 \quad (3)$$

where  $f_{out}$  denotes the output feature map size,  $f_{in}$  represents the input feature map size,  $k$  indicates the size of the convolutional kernel, and  $s$  stands for the stride size.

### 3.2.2. Focal Modulation

Since the Vision Transformer is launched in the field of the Computer Vision, the potential of it is already proven in

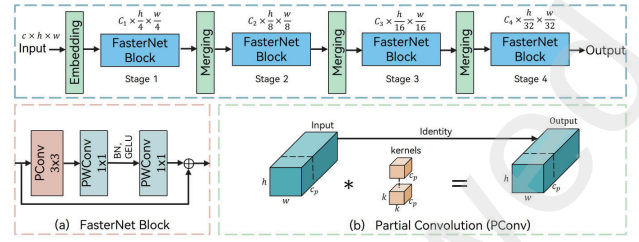


Figure 5: The network structure of FasterNet.

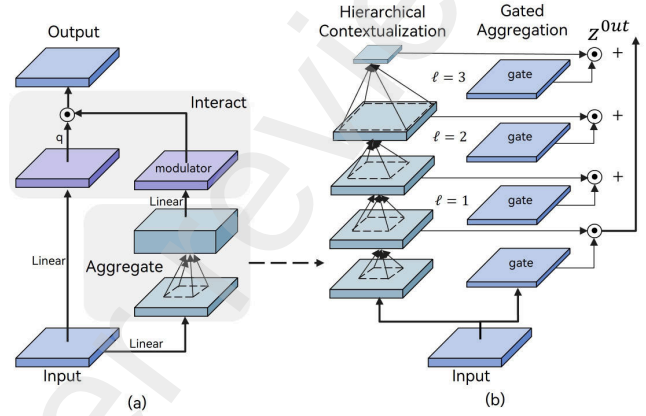


Figure 6: (a) Focal Modulation; (b) Context aggregation

practice in numerous vision tasks. This innovative architecture shows remarkable results in dealing with visual challenges and is gradually becoming one of the key technologies in the field of the Computer Vision. The Self-Attention (SA), which is the main core of Vision Transformer, is able to deal with long distance dependencies. It means that it is able to establish a connection between any two positions of an image. However, this causes efficiency problems because the SA needs to compute the relationship between all pairs of pixel positions in the input images, thus leads to a relatively large amount of computation when processing large size images (Guo et al., 2023). For this problem, as illustrated in Figure 6 (a) the Focal Modulation (FM) first aggregates the context of the input features, and then the contextual information is injected into the query tokens in the form of modulation. This differentiates from the SA mechanism, the Focal Modulation improves processing speed and accuracy by aggregating contextual features earlier and then performing lightweight interaction operations to build attention mechanism-free architectures.

Figure 6 (b) illustrates the hierarchical context aggregation process of FM, which consists of two steps: The Hierarchical Contextualization, which extracts context from local to global scales at different levels of granularity; The Gated Aggregation, which compresses all the contextual features at different levels of granularity into a modulator. This stage produces a set of re-weighted and adjusted features that provide more accurate and effective feature information for subsequent processing stages.

- (1) In the Hierarchical Contextualization step, for the input feature map  $X$ , it is first projected into a new feature space through a linear layer, resulting in  $Z^0 = f_z(X) \in \mathbb{R}^{H \times W \times C}$ . Then, a hierarchical context representation is obtained by stacking  $L$  layers of depth-wise convolutions. At each focal layer level  $\ell \in \{1, \dots, L\}$ , its output  $Z^\ell$  is acquired as follows:

$$Z^\ell = f_a^\ell(Z^{\ell-1}) \triangleq \text{GeLU}(\text{DWConv}(Z^{\ell-1})) \quad (4)$$

$f_a^\ell$  stands for the context function at the  $\ell$  level, which is realized by the deep-wise convolution (DWConv) with a convolutional kernel size of  $k^\ell$  and the GeLU activation function. To capture the global context of the entire input, the global average pooling is applied on the feature map at layer  $L$ . Specifically,  $L$  performs average pooling on the layer's feature maps  $Z^L$  to get  $Z^{(L+1)} = \text{AvgPool}(Z^L)$ , and obtains a total of  $L+1$  feature maps. It is a process that helps to extract and utilize layer-specific feature information while maintaining computational efficiency.

- (2) In the Gated Aggregation step, the  $L+1$  feature maps obtained from the Hierarchical Contextualization are condensed into a single modulator. In images, the relationship between a visual token (query) and its surrounding context usually depends on the content itself. For example, a model may rely on local fine-grained features to highlight queries to visual objects, but primarily make use of global coarse-grained features to handle queries to background scenes. Based on this situation, the FM uses a gating mechanism to control the extent to which each query is aggregated from different levels, and this flexible aggregation helps to improve the efficiency and performance of the model. Specifically, a linear layer is used to obtain spatial and level-aware gating weights  $G = f_g(X) \in \mathbb{R}^{H \times W \times (L+1)}$ , where  $G^\ell$  is the slice of  $G$  at level  $\ell$ . Then, a weighted summation is performed by element-by-element multiplication to obtain a single feature  $Z^{\text{out}}$  of the same size as the input  $X$ , where

$$Z^{\text{out}} = \sum_{i=\ell}^{L+1} G^\ell \cdot Z^\ell \quad (5)$$

### 3.2.3. KernelWarehouse

The conventional dynamic convolution works by learning a linear mixture of static kernels, but these methods are not efficient and significantly increase the number of convolution parameters (Li et al., 2024a). However, the Towards Parameter-Efficient Dynamic Convolution Kernel-Warehouse (KWConv) employs both the Kernel Partition and the Warehouse Sharing strategies. Inspired by the following two aspects, KWConv redefines "kernel" and "assembled kernel" in dynamic convolution from the perspective of reducing kernel dimensions and increasing the number of kernels to solve the above problems. On the one hand, the current dynamic convolution methods usually treat all the parameters of a standard convolutional layer as a whole kernel and increase the number of kernels to  $n$ . Then, the

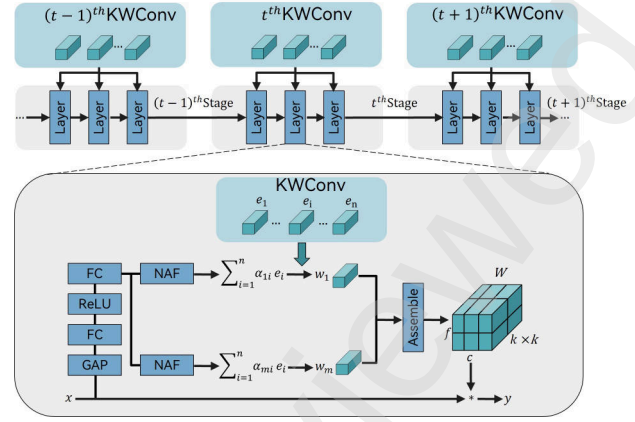


Figure 7: The workflow of the KernelWarehouse.

kernels are fused into a linear mixed kernel using an attentional mechanism. This approach is simple and effective, but it fails to take into account the parameter dependencies within a single static kernel in a convolutional layer. On the other hand, the existing dynamic convolution strategies deploy  $n$  kernels independently for each layer of a convolutional neural network, but ignore the parameter dependencies between different layers. However, the KWConv can provide a good balance between parametric efficiency and characterization ability based on the above inspiration.

Figure 7 presents the workflow of the KWConv method, which consists of three key components: the Kernel Partition, the Warehouse Sharing, and an Attention Function. For an input feature map  $x$ , it first undergoes a global average pooling (GAP) operation, which compresses the spatial information of each channel to generate a global feature vector. This vector is then further processed through two layers of a Fully Connected (FC) network: the first FC layer aims to reduce the length of the feature vector, while the second FC layer generates a set of features for each potential kernel unit. The processed features are delivered to an Attention Function (NAF) through the ReLU activation function. The NAF dynamically calculates a series of scalar attention weights ( $\alpha_1, \dots, \alpha_m$ ) based on the input features. And these weights are used to determine the degree of contribution of different kernel units in the final composition of the dynamic convolutional kernel. The calculation formula for the NAF is

$$\alpha_{ij} = (1 - \tau) \frac{z_{ij}}{\sum_{p=1}^n |z_{ip}|} + \tau \beta_{ij}, j \in \{1, \dots, n\} \quad (6)$$

here,  $\tau$  is a temperature parameter that linearly decreases from 1 to 0 during the early stages of training;  $z_{ij}/(\sum_{p=1}^n |z_{ip}|)$  is a linear normalization function capable of generating negative attention outputs;  $\beta_{ij}$  is a binary value (0 or 1) that is used to initialize attention.

The attention weights obtained above are linearly combined with the kernel units ( $e_1, \dots, e_n$ ) predefined in the warehouse to generate a series of dynamic convolutional kernel units ( $w_1, \dots, w_m$ ). Each dynamic convolutional kernel unit is a weighted sum of the warehouse kernel units whose

weights are given by the scalar attention described above. Where the kernel unit is defined as

$$w_i = \alpha_{i1}e_1 + \dots + \alpha_{in}e_n, i \in \{1, \dots, m\} \quad (7)$$

and these dynamic convolution kernel units are assembled into the final dynamic convolution kernel  $W$ . This is the strategy of the Kernel Partition, where the static kernel  $W$  of a regular convolutional layer is sequentially divided into  $m$  disjoint parts  $(w_1, \dots, w_m)$ . The dynamic convolution kernel  $W$  is then utilized to perform a convolution operation to map the input feature map  $x$  to the output feature map  $y$ . This process enables the network to dynamically adjust its feature extraction methods based on the input, thereby enhancing adaptability and efficiency. The bottom of Figure 7 demonstrates the Warehouse Sharing strategy, which shows that multiple convolutional layers in the same stage can use the kernel units from the same warehouse. This design not only reduces the total number of model parameters but also ensures effective sharing and transfer of feature information across different layers of the network.

### 3.3. Apple counting method

It is extremely important to reduce errors in apple detection and counting in order to improve the accuracy of the apple yield estimates. In the real-time detection process, the multi-target tracking (MOT) can effectively distinguish and track multiple objects in a video, realizing the continuous identification and the position tracking of each target. However, it is easy to duplicate tracking for targets that are too similar. Therefore, the multi-object tracking algorithm DeepOC-SORT builds on motion-based MOT algorithms by incorporating appearance information of targets to address issues caused by occlusions, motion blur, or objects with similar appearances.

Figure 8 illustrates the tracking process of the Deep OC-SORT. The apples position and feature information that passes through the apple detection method first passes through the Camera Motion Compensation (CMC) module to correct the visual error caused by the camera movement. Simultaneously, The Re-identification (ReID) module extracts the key appearance features of the apples from the images to distinguish between different apple individuals. After the two parts of information are merged, they are processed by the OC-SORT's Kalman Filter (KF) to predict the future state of the apple and update its current state. Then, the adaptive weighting (AW) module adjusts the weight of each feature according to the differentiation ability of the apple features. Finally, the Cost Matrix and the Linear Solver work in tandem to solve the data correlation problem, and ensure that the same apples are accurately matched and tracked in a sequence of the video frames. The high degree of synergy in the whole process enables the Deep OC-SORT to efficiently track every detected apple objects in a dynamic environment and to assign a uniquely identified ID number for every apple.

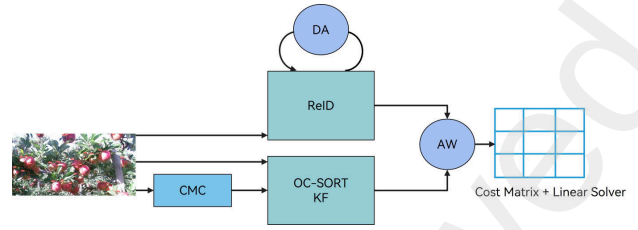


Figure 8: The tracking processes of the Deep OC-SORT.

## 4. Results and discussion

### 4.1. Experimental platform and evaluation index

Since the YOLOv8 based architecture is utilised in the experiments, the YOLOv8 is served as the baseline model for the experiments. All the comparison methods involved in the experiments are based on the YOLOv8 architecture. For ensuring the experimental results are reproducible and comparable, all experiments are conducted in a unified environment. The experimental platform uses the Windows 11 operating system and is equipped with an Nvidia Geforce RTX 3090 GPU with 24GB of video memory. The programming language for the model is Python 3.8 and uses the Pytorch version 2.0.0 deep learning framework and the CUDA 11.7 computational architecture. In the process of model training, the uniformly set parameters include: training period (epochs) of 300, batch size of 16, input image size (imgsz) of 640, and the use of Adam as the optimizer. The uniform configuration of these parameters is intended to ensure standardization of the training process so that the results can be effectively compared and analyzed.

In the experimental process, the Precision, Recall, and mean Average Precision (mAP) are adopted to evaluate the performance of the apple yield estimation model. In addition, the performance representation of the model is also monitored, including the number of parameters, and the speed of detection. The calculation methods for the Precision and the Recall are as follows

$$\text{Precision} = \frac{TP}{TP + FP} \quad (8)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (9)$$

where the True Positive (TP) represents the number of positive samples correctly identified as positive; the False Positive (FP) stands for the number of negative samples incorrectly identified as positive; and the False Negative (FN) is the number of positive samples incorrectly identified as negative. The Precision reflects the model's ability to correctly identify positive samples, so a high Precision means that there are fewer false positives in the samples that the model marks as positive. The Recall reflects the proportion of all actual positive samples that are correctly recognized as positive by the model. A high Recall indicates that the model is able to capture more positive samples and reduce under-reporting. However, the formula for calculating the mAP is

as follows

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (10)$$

Where,  $N$  denotes the number of the detection categories (the number of the categories in the experiment is 1);  $AP_i$  represents the Average Precision (AP) of the  $i$ th category, and it is derived by calculating the area under the Precision-Recall curve for the category. The mAP is the AP value of all categories totaled and divided by the number of categories, and reflects the comprehensive performance of the model. In the mAP, it is usually combined with one or more the Intersection over Union (IoU) thresholds so that the degree of overlap between the model-predicted bounding box and the true bounding box can be measured. For instance, the mAP50 considers the detection frames with at least 50% overlap, while the mAP50-95 considers the average performance at the different IoU thresholds from 50% to 95%. If the mAP with a high threshold is used, its detection accuracy is lower, which means that the detection is more difficult. However, during the experiments, improving the accuracy at the high thresholds so that the model can locate the objects more accurately and improve the robustness of the model is a priority.

#### 4.2. Performance Analysis of the Input Sizes

We explore the impact of three image sizes, 320x320, 640x640 and 1280x1280, on the model performance in order to find a suitable image input size for limited resources. Figure 9 displays the GPU resource consumption over 300 training epochs for three different image sizes, denoted by GPU\_MEM. Table 2 describes in detail the performance of the three sizes of images in terms of the mAP performance and the mean GPU\_MEM. The results show that when the size is increased, the mAP value is also increased. The reason for this is that the large size of the input images provides more detailed information, which facilitates the model to accurately detect subtle features. However, this boost is not always proportional. For example, while the 1280x1280 size image is the highest mAP, its the mean GPU\_MEM consumption is a whopping 20.9G, almost four times that of the 640x640 size. By considering the balance between performance and resource consumption, 640x640 size is chosen as the input for the model in the experiment. This selection not only considers the mAP performance (4.9% and 12.7% improvement for the 640x640 mAP50 and mAP50-95, respectively, compared to the 320x320 size), but also the rational utilization of GPU resources. For overall consideration, the 640x640 size is more suitable for the needs of the AppleYOLO because it ensures higher detection accuracy while avoids excessive resource consumption.

#### 4.3. Sample Augmentation Analysis

In response to the small experimental samples, the samples are expanded during the experiments. However, to verify whether the sample expansion is favorable to the experimental results, two performance metrics, the mAP50 and

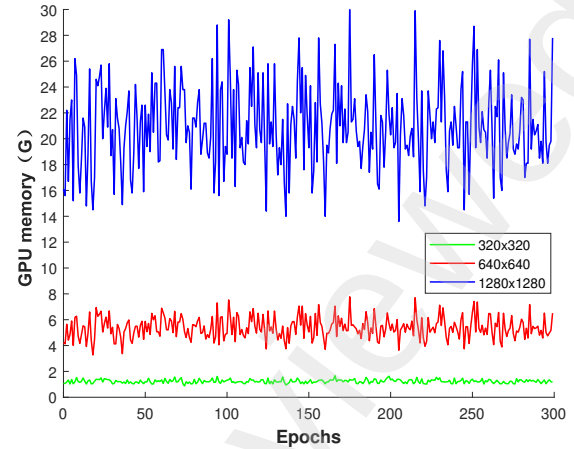


Figure 9: Impact of input image sizes on GPU resource usage.

Table 2

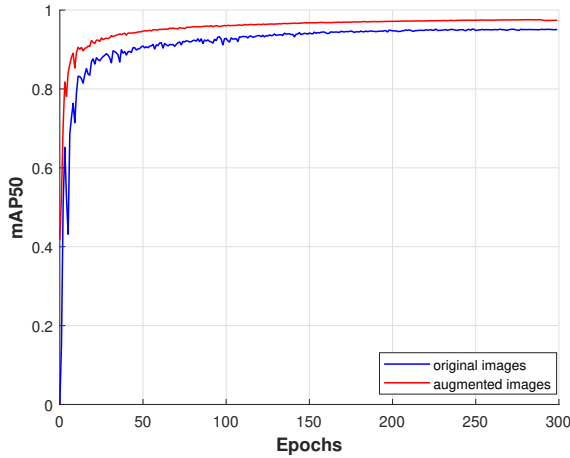
Detailed performance parameters for different input image sizes.

| inputSize | mAP50 | mAP50-95 | m(GPU_MEM) |
|-----------|-------|----------|------------|
| 320x320   | 0.926 | 0.62     | 1.23G      |
| 640x640   | 0.975 | 0.747    | 5.38G      |
| 1280x1280 | 0.986 | 0.796    | 20.9G      |

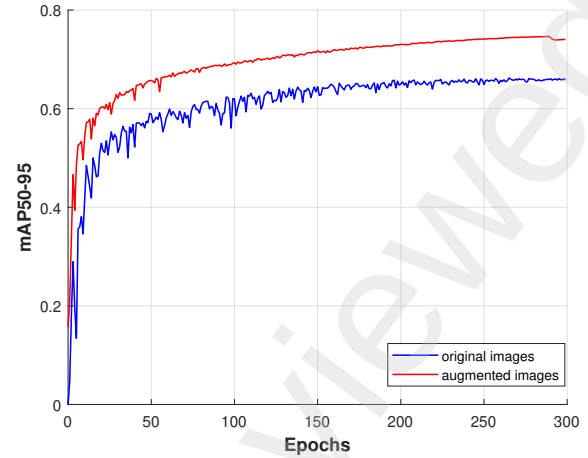
the mAP50-95 are compared. Figure 10 demonstrates the performance of mAP50, where the original images improve rapidly in the initial phase and gradually stabilize. However the augmented images show a more steady boost and beyond the original images after about 20 epochs. The performance of the augmented images remains slightly higher than the original images until the end of 300 epochs. Figure 11 exhibits the performance of the mAP50-95, with more significant differences between the original and augmented images. The performance of the augmented images steadily increases and remains ahead throughout the experiment. The experiments show that taking a sample augmentation approach has a significant positive impact on the overall performance of the model.

#### 4.4. Performance evaluation of Focal Modulation

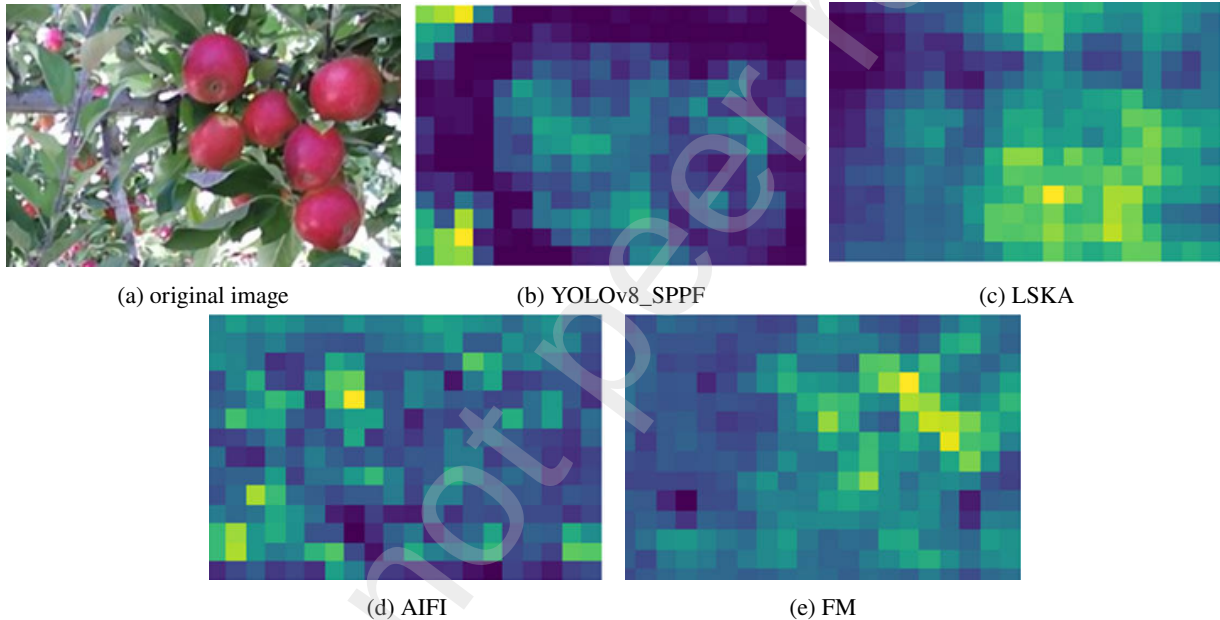
The feature aggregation module FM of AppleYOLO plays an important role in the whole model. By relying on FM, the contextual information can be effectively aggregated to capture the key information in the image and mitigate the effects of different scale sizes. For verification the effectiveness of the FM, it is compared with several feature aggregation mechanisms, such as SPPF, LSKA, and AIFI. Among them, the SPPF is used as a pooling layer for the YOLOv8, which adaptively fuses information from different scales; The LSKA is a Large Separable Kernel Attention mechanism that can handle the long-range dependency problem (Lau et al.,2024); The AIFI is an Attention-based Intra-scale Feature Interaction mechanism is used to overcome the problem of accuracy and computational efficiency



**Figure 10:** The influence of sample augmentation on mAP50.



**Figure 11:** The influence of sample augmentation on mAP50-95.



**Figure 12:** FM and other methods for visualising feature maps.

in object detection and recognition (Lv et al., 2023). Figure 12 displays the feature maps of the apple image after four layers of feature aggregation mechanisms respectively, and the results indicate that the FM is better able to identify the spatial location of the apples and capture the key information. Figure 13 displays the performance comparison of the four different methods over 300 training epochs, with mAP50-95 as the performance metric.

Table 3 summarizes the detailed performance parameters of the FM in comparison with other three mechanisms. On the important detection metric mAP0.5-0.95, the FM leads with 77.1% (a 2.4% improvement over YOLOv8's SPPF) and the mAP50 reaches 98%. Meanwhile, the FM also achieves the highest scores of 95.8% and 94.5% for precision and recall, respectively. Furthermore, the number of parameters and the amount of computation (GFLOPS) of the

FM are comparable to the benchmark model, and the performance in terms of frame rate (FPS) is close to optimal. It means that the FM is both efficient and practical in real-world applications. As a whole, these parameters prove the superior performance of the FM in all aspects.

#### 4.5. Performance evaluation of KernelWarehouse

The KWConv is applied to the feature fusion part of the AppleYOLO. In order to prove that the KWConv can indeed be advantageous in AppleYOLO, it is compared to other convolutions and a series of comparison experiments are set up. Among them, the feature fusion part of the YOLOv8 model uses the c2f to improve the representation of features at different scales. The Diverse Branch Block (DBB) enriches the feature space by combining branches of different sizes and complexities to enhance the representation

**Table 3**

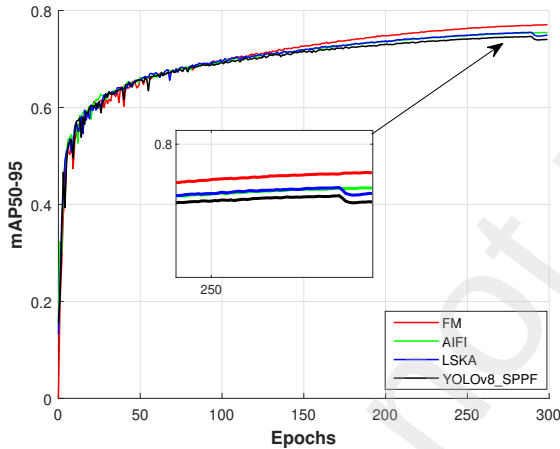
Different characteristics of aggregation mechanism performance parameters (Setting the batch to 1 when calculating FPS).

| Name        | Size | Precision(%) | Recall(%)    | mAP0.5(%)   | mAP0.5:0.95(%) | Parameters/M | GFLOPS | FPS(b@1)     |
|-------------|------|--------------|--------------|-------------|----------------|--------------|--------|--------------|
| YOLOv8_SPPF | 640  | 0.955        | 0.935        | 0.975       | 0.747          | 3.01         | 8.1    | 112.4        |
| LSKA        | 640  | 0.957        | 0.94         | 0.978       | 0.755          | 3.23         | 8.3    | 109.5        |
| AIFI        | 640  | 0.956        | 0.938        | 0.978       | 0.751          | 2.93         | 8.0    | 89.7         |
| <b>FM</b>   | 640  | <b>0.958</b> | <b>0.945</b> | <b>0.98</b> | <b>0.771</b>   | 3.11         | 8.2    | <b>110.2</b> |

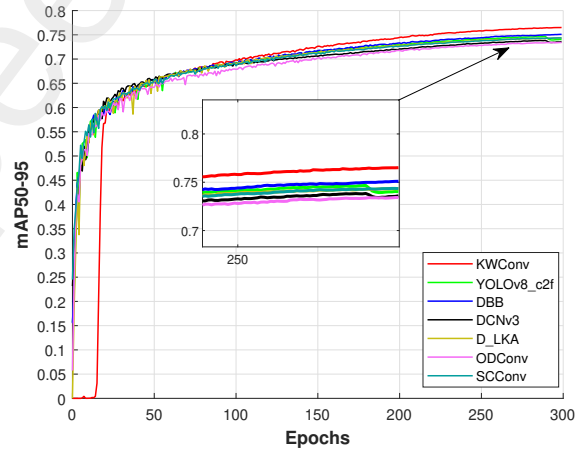
**Table 4**

Different convolution performance parameters (Setting the batch to 1 when calculating FPS).

| Name          | Size | Precision(%) | Recall(%)    | mAP0.5(%)    | mAP0.5:0.95(%) | Parameters/M | GFLOPS     | FPS(b@1)     |
|---------------|------|--------------|--------------|--------------|----------------|--------------|------------|--------------|
| YOLOv8_C2f    | 640  | 0.955        | 0.935        | 0.975        | 0.747          | 3.01         | 8.1        | 112.4        |
| DBB           | 640  | 0.954        | 0.94         | 0.977        | 0.751          | 3.01         | 8.1        | 53.8         |
| DCNv3         | 640  | 0.952        | 0.935        | 0.974        | 0.742          | 2.84         | 7.8        | 52.5         |
| SCConv        | 640  | 0.952        | 0.933        | 0.975        | 0.744          | 3.95         | 9.3        | 55.3         |
| ODConv        | 640  | 0.954        | 0.928        | 0.972        | 0.734          | 3.07         | 5.7        | 50.9         |
| D_LKA         | 640  | 0.954        | 0.933        | 0.974        | 0.741          | 3.67         | 8.6        | 48.4         |
| <b>KWConv</b> | 640  | <b>0.958</b> | <b>0.942</b> | <b>0.979</b> | <b>0.764</b>   | 3.09         | <b>4.5</b> | <b>101.2</b> |



**Figure 13:** The performance comparison of the FM with other feature aggregation mechanisms.

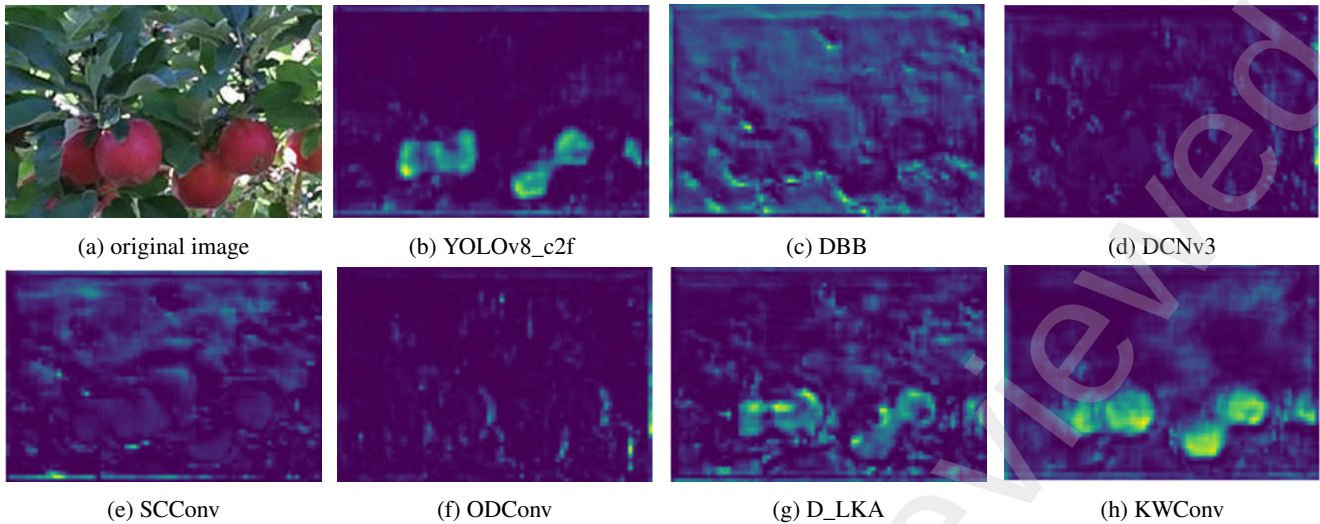


**Figure 14:** The performance comparison of different convolutions.

ability of a single convolution. It can improve network model performance without any inference time cost (Ding et al., 2021). The Deformable Convolution v3 (DCNv3) adopts a weight sharing strategy similar to regular convolution, and this strategy significantly reduces the number of parameters and memory usage for large models. Moreover, it achieves sparse sampling by dividing the convolutional weights into a depth-dependent part and a point-wise convolutional part. (Wang et al., 2023b). The Self-Calibrated Convolution (SCConv) exploits spatial and channel redundancy between features for CNN compression to reduce redundant computation and facilitate learning of representative features (Liu et al., 2020). The Omni-dimensional Dynamic Convolution (ODConv) leverages a novel multi-dimensional atten-

tion mechanism and parallel strategy to learn complementary attention of the convolution kernels across all four dimensions of the kernel space in any convolution layer (Li et al., 2022b). As an alternative to regular convolutions, the ODConv can be inserted into many CNN architectures. The Deformable Large Kernel Attention (D\_LKA) uses a large convolutional kernel to fully understand the context and flexibly transforms the sampling networks, thus enabling the model to adapt to different data patterns and features (Azad et al., 2023).

Figure 14 presents the mAP50-95 performance metric of KWConv compared to other convolutions. In the initial few epochs, the metric for the KWConv start at 0. The reason for this is analysed and find that it is due to the relative complexity of the structure of the KWConv, which may re-



**Figure 15:** KWConv and other methods for visualising feature maps.

quire more time for the model to learn an effective feature representation in the early stages of training. Nevertheless, after 100 epochs, the accuracy of the KWConv gradually surpasses other convolutions, showing optimal and stabilizing performance. The results of feature extraction at the feature fusion layer for KWConv and several other convolutions respectively are shown in Figure 15. Table 4 provides a detailed comparison of the performance between KWConv and other convolutional models. The KWConv's performance is particularly notable, with its mAP50 reaching 97.9%, and mAP50-95 reaching 76.4%. In comparison to the baseline model, mAP50-95 improved by 1.7%. With a similar number of parameters, its computational cost in GFLOPS is reduced by 3.6 compared to the baseline model. The results indicate that the KWConv is both efficient and accurate as a convolutional network. Although it starts slower in the initial training phase, it ultimately achieves high performance while significantly reducing computational costs. Since the KWConv shows the ability to maintain high accuracy at low computational cost, it provides greater flexibility to explore more complex backbone network architectures.

#### 4.6. Performance evaluation of FasterNet

The backbone of the AppleYOLO is the foundation of the entire network model, like the frame of a building. It is responsible for extracting and learning shallow features from images, and the learned features will directly affect the subsequent layers. Also, the performance of the backbone network directly affects the overall performance of the network. A powerful backbone can capture rich feature representations that help improve the accuracy and robustness of the whole network. In the AppleYOLO, the FasterNet is employed as the backbone network. To verify that the FasterNet can significantly improve the performance of the AppleYOLO compared to other neural network architectures, a series of comparative experiments are designed. Among the compared models, the Darknet53 is used as the backbone

by YOLOv8; The EfficientViT is an efficient CNN model similar to FasterNet (Liu et al., 2023); And Swin Transformer is a model based on the Transformer architecture (Liu et al., 2021).

Figure 16 plots the confusion matrix when each of the four methods is used as the backbone of the model. In comparison FasterNet has the lowest probability of incorrectly identifying apples as background. Figure 17 shows the change in the mAP50-95 performance metric with training epochs for the different methods. Table 5 details the performance parameters of several deep neural networks. It can be observed that the FasterNet as the backbone of the network model outperforms the benchmark model in terms of accuracy, with a 1.3% improvement in the mAP50-95. Although FasterNet's computational volume is increased by 2.6, this increase is within acceptable limits. In addition, the low-computation nature of the KWConv helps to compensate for the slightly higher computation requirements of the FasterNet, and realizes a balance between total performance and efficiency. However, compared to the EfficientViT, the FasterNet has better overall performance. Although the Swin Transformer has improved accuracy compared to the benchmark model, its number of parameters and computational complexity are as high as 29.9 and 79.1, respectively. This is approximately 10 times the benchmark model. Considering the accuracy as well as the number of parameters and the amount of computation, the FasterNet performs better in the AppleYOLO.

#### 4.7. Ablation Study

To comprehensively evaluate the validity and performance of the AppleYOLO detection, a series of ablation experiments are designed to focus on the three core innovations of FasterNet, FM and KWConv. By fusing these modules sequentially in the detection model, it is possible to observe the impact of each fusion on the overall performance of the model. This leads to a deeper understanding of the combined

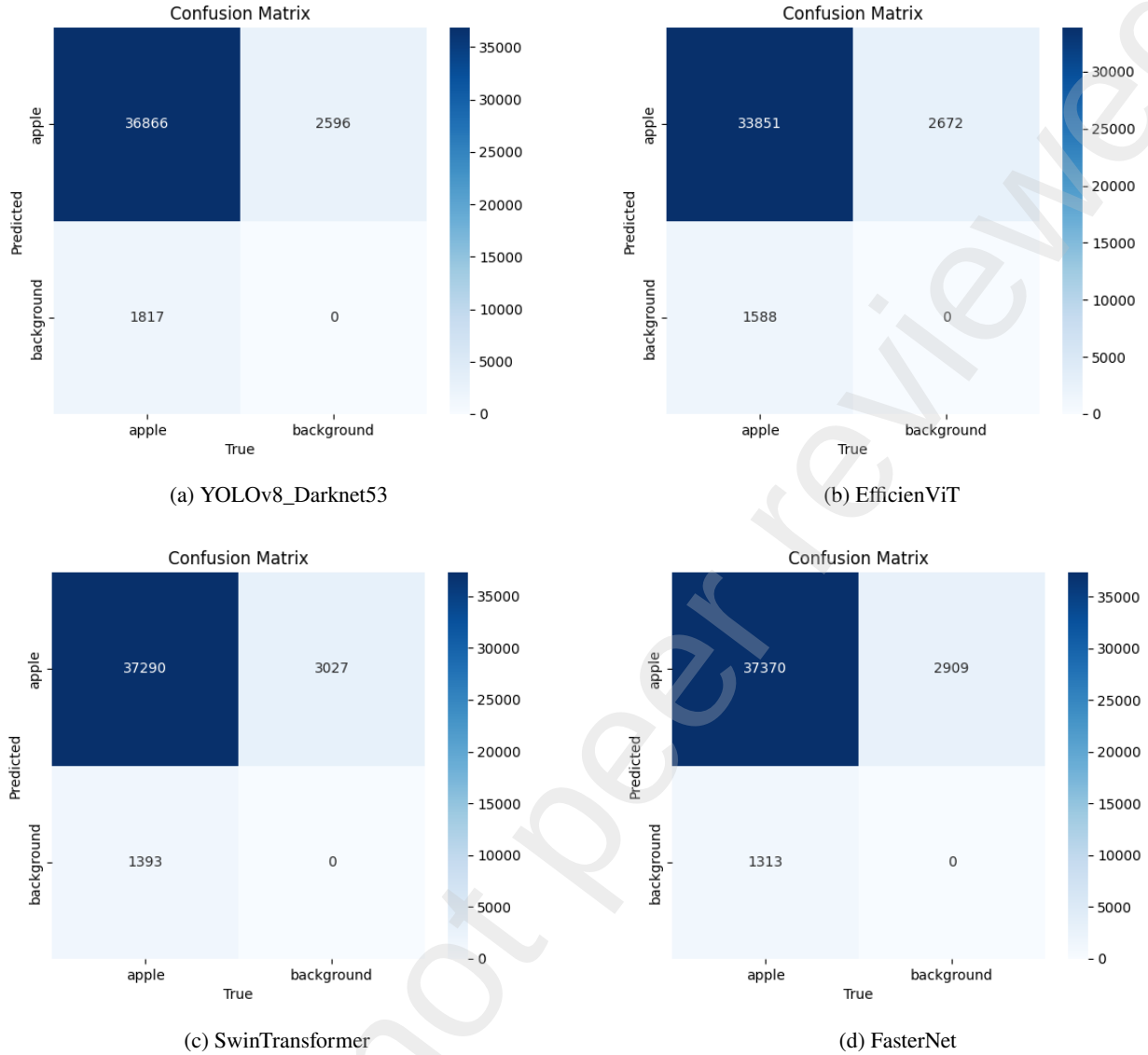


Figure 16: FasterNet and other methods for visualising feature maps.

improvement in detection performance of the AppleYOLO when they work together.

Figure 18 illustrates the effect of sequentially merging the three modules on the performance of the mAP50-95. When compared to the baseline model, the mAP50-95 values increased after each fusion of new modules. In the end, the best performance is achieved by completely merging the three modules, and the model gradually stabilises during training. Table 6 contains the detailed results of the ablation experiments. Although the FasterNet and the FM increase the computational amount slightly (by about 2.7), their impact on the overall performance is still within a reasonable range. With the subsequent addition of the KW-Conv, it effectively balances the higher computational and parametric quantities of the previous model due to its improved parametric efficiency. And the computational and parametric quantities only increased by 1.42 and 1.6 relative

to the baseline model, respectively. Other than that, the introduction of the KWConv significantly improves the performance of the AppleYOLO on mAP50 and mAP50-95 by 1% and 5.1% respectively compared to the baseline model. Figure 19 present the visualisation of the ablation experiments and the results confirm that the introduction of the three core modules indeed improves the detection.

#### 4.8. Comparison experiments with state-of-the-art methods

We compare the AppleYOLO detection performance with the current state-of-the-art deep learning methods (seven methods methods in Table 7). Figure 21 illustrates the mAP50-95 performance comparison between the AppleYOLO and other seven deep learning methods. With an increase in training epochs, the AppleYOLO gradually shows significant superiority in this performance metric over oth-

**Table 5**

Detailed performance parameters of different depth neural networks (Setting the batch to 1 when calculating FPS).

| Name             | Size | Precision(%) | Recall(%)    | mAP0.5(%)   | mAP0.5:0.95(%) | Parameters/M | GFLOPS      | FPS(b@1)    |
|------------------|------|--------------|--------------|-------------|----------------|--------------|-------------|-------------|
| YOLOv8_Darknet53 | 640  | 0.955        | 0.935        | 0.975       | 0.747          | 3.01         | 8.1         | 112.4       |
| EfficienViT      | 640  | 0.957        | 0.935        | 0.976       | 0.743          | 4.01         | 9.4         | 37.5        |
| SwinTransformer  | 640  | 0.952        | 0.943        | 0.98        | 0.753          | 29.9         | 79.1        | 47.3        |
| <b>FasterNet</b> | 640  | <b>0.958</b> | <b>0.946</b> | <b>0.98</b> | <b>0.76</b>    | <b>4.17</b>  | <b>10.7</b> | <b>96.9</b> |

**Table 6**

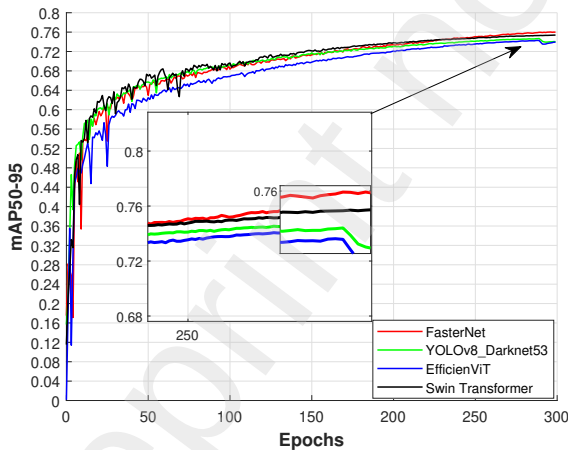
Detailed performance parameters in ablation experiment (Setting the batch to 1 when calculating FPS).

| Name                       | Precision(%)    | Recall(%)       | mAP0.5(%)       | mAP0.5:0.95(%)  | Parameters/M   | GFLOPS     | FPS(b@1)     |
|----------------------------|-----------------|-----------------|-----------------|-----------------|----------------|------------|--------------|
| YOLOv8                     | 0.955           | 0.935           | 0.975           | 0.747           | 3.01           | 8.1        | 112.4        |
| YOLOv8+FasterNet           | 0.958           | 0.946           | 0.98            | 0.76            | 4.17           | 10.7       | 96.9         |
| YOLOv8+FasterNet+FM        | 0.961           | 0.952           | 0.982           | 0.78            | 5.23           | 10.8       | 99.6         |
| YOLOv8+FasterNet+FM+KWConv | <b>0.966(↑)</b> | <b>0.958(↑)</b> | <b>0.985(↑)</b> | <b>0.798(↑)</b> | <b>4.43(↑)</b> | <b>9.7</b> | <b>100.3</b> |

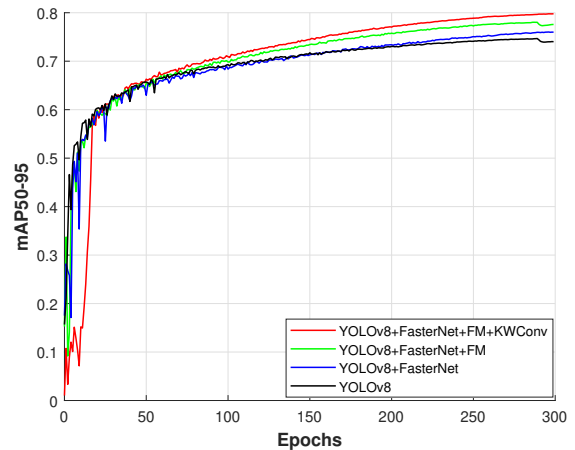
**Table 7**

Detailed performance parameters of different deep learning methods (Setting the batch to 1 when calculating FPS).

| Name             | Size | Precision(%) | Recall(%)    | mAP0.5(%)    | mAP0.5:0.95(%) | Parameters/M | GFLOPS     | FPS(b@1)     |
|------------------|------|--------------|--------------|--------------|----------------|--------------|------------|--------------|
| YOLOv3-tiny      | 640  | 0.959        | 0.93         | 0.97         | 0.739          | 12.13        | 18.9       | 291.3        |
| YOLOv5           | 640  | 0.947        | 0.929        | 0.972        | 0.728          | 2.5          | 7.1        | 94.8         |
| YOLOv6           | 640  | 0.944        | 0.919        | 0.968        | 0.72           | 4.23         | 11.8       | 122.1        |
| YOLOv7           | 640  | 0.96         | 0.97         | 0.987        | 0.726          | 37.2         | 105.1      | 35.5         |
| YOLOv8           | 640  | 0.955        | 0.935        | 0.975        | 0.747          | 3.01         | 8.1        | 112.4        |
| RT-DETR          | 640  | 0.955        | 0.943        | 0.978        | 0.738          | 9.48         | 16.7       | 43.6         |
| YOLOv9-c         | 640  | 0.965        | 0.957        | 0.986        | 0.8            | 51           | 238.9      | 39           |
| <b>AppleYOLO</b> | 640  | <b>0.966</b> | <b>0.958</b> | <b>0.985</b> | <b>0.798</b>   | <b>4.43</b>  | <b>9.7</b> | <b>100.3</b> |



**Figure 17:** The FasterNet and other deep neural network performance parameters.

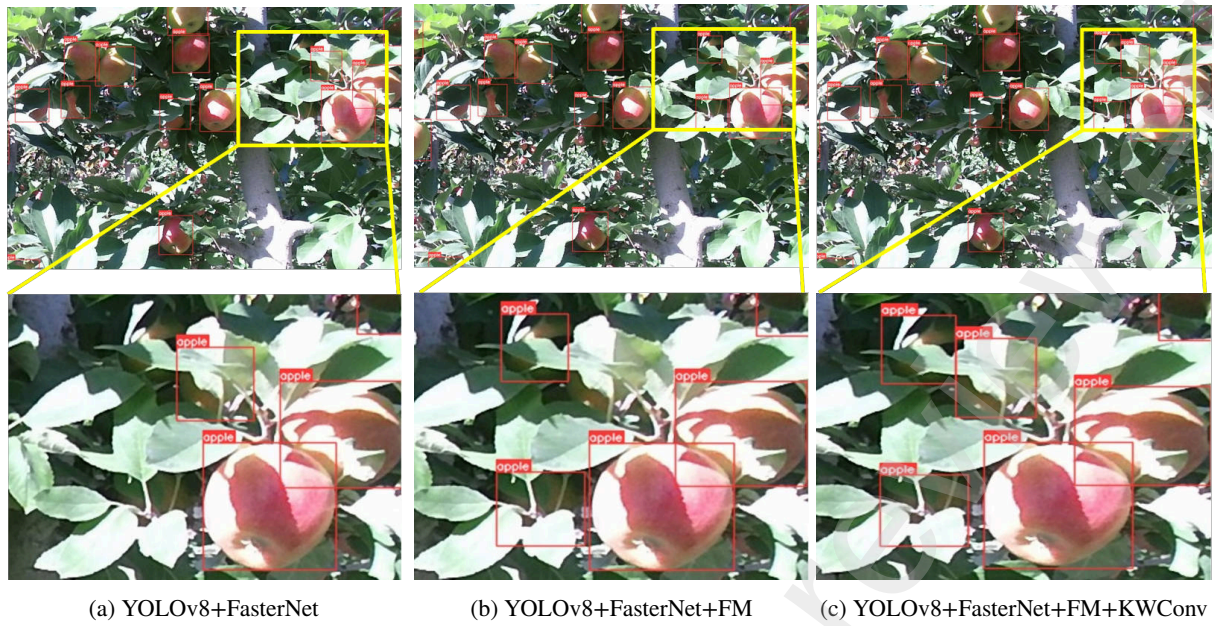


**Figure 18:** The results of the apple detection model ablation experiment.

ers. In particular, the AppleYOLO performance surpasses the latest YOLOv9-c (Wang and Liao, 2024) and RT-DETR (Lv et al., 2023).

Table 7 details the performance parameters of the Ap-

pleYOLO in comparison with seven other methods. Although the mAP50 of the YOLOv3-tiny (Redmon and Farhadi, 2018) is close to the mAP50 of the AppleYOLO, its the mAP50-95 difference is 5.9%. Moreover, the YOLOv3-



**Figure 19:** Visualization results of ablation experiments.

tiny has a much larger number of parameters and computations. However, the performance parameters of the AppleYOLO are basically optimal compared to the YOLOv5 (Li et al., 2023) and YOLOv6 (Li et al., 2022a). In particular, while the YOLOv7 (Wang et al., 2023a) gives good results in terms of the Precision, Recall, and mAP, its large number of parameters and computation also imply a high consumption of computational resources. It means that the YOLOv7 may not be the most ideal option in resource-constrained environments. Then, compared to the benchmark method YOLOv8, the AppleYOLO's mAP50 and mAP50-95 are improved by 1% and 5.1%, respectively. And the number of parameters and computation are similar to the YOLOv8. Besides, the AppleYOLO maintains a similar level of mAP compared to the latest large-scale model YOLOv9-c, but the AppleYOLO significantly outperforms the YOLOv9-c in terms of the number of parameters, the amount of computation, and the processing speed. Figure 20 displays a visualisation of some of the comparative experimental results. In contrast to AppleYOLO, several other methods performed poorly in at least one of the following three cases: accurately detecting apples obscured by leaves; accurately identifying apples that overlap each other; and accurately discriminating between green leaves and green apples.

#### 4.9. Evaluation of the counting method

To evaluate the impact of the counting algorithm Deep OC-SORT on the apple yield estimation method AppleYOLO, 49 apple images are selected for testing. The number of apples in each image is manually labelled as a standard to compare and validate the counting results of the algorithm. Figure 22 presents the regression analysis between algorithmic counting and manual counting. A strong correlation between algorithmic counting and manual counting

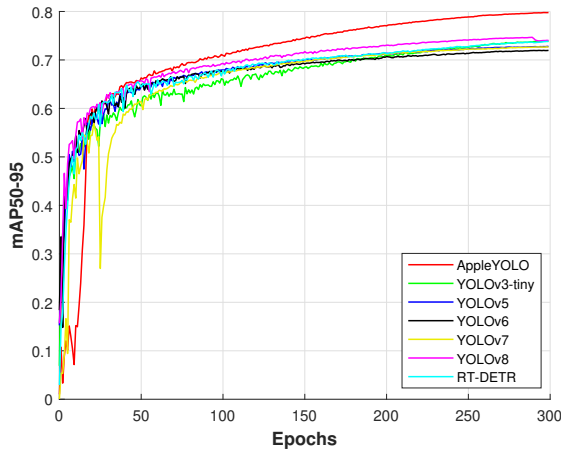
can be clearly observed ( $R^2 = 0.95$ ). It shows that the AppleYOLO can accurately reflect the actual number of apples in the vast majority of cases. Figure 23 displays that the difference between the algorithm counting and the manual counting is mainly focused around zero. This indicates that the algorithm counting is typically very close to the manual counting. Specifically, more than 90% of the images show a counting error within an acceptable range (less than or equal to 6). This reveals that the apple counting method is not only reliable but also highly accurate, making it potentially directly applicable in the agricultural field for real-time estimation of apple yield.

## 5. Conclusion

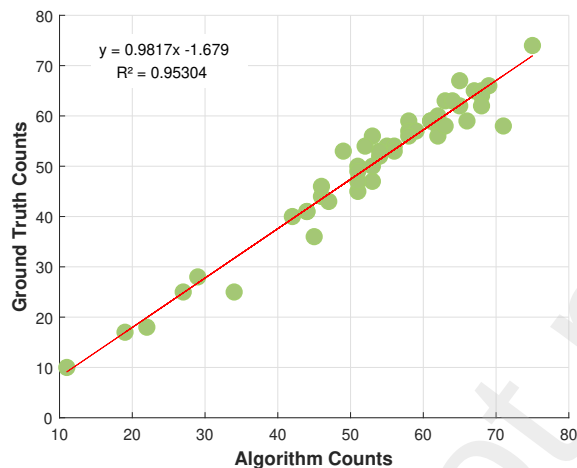
To accurately estimate apple yield in complex orchard environments, an Apple yield estimate method based on YOLOv8 and Deep OC-SORT (AppleYOLO) is proposed. The method enhance the ability to extract apple feature information by merging several advanced techniques. Including the FasterNet, a fast neural network to fully extract apple edge messages; The Focal Modulation, a feature aggregation strategy for enhanced feature representation (apple size and location); And the KernelWarehouse, a Towards Parameter-Efficient Dynamic Convolution technique to improve the ability of learning complex features. Additionally, a new method for apple counting based on motion analysis called Deep OC-SORT is introduced in AppleYOLO, and this method exhibits significant results in improving the accuracy of apple counting. The experimental results suggest that the AppleYOLO achieves a high accuracy of 98.5% on the standard detection metric mAP50, as well as 79.8% on the more stringent mAP50-95 metric. And compared with the benchmark model, it improves by 1% and 5.1%, respec-



**Figure 20:** Visualization of some comparative experimental results.



**Figure 21:** The AppleYOLO detection performance comparison with different deep learning methods.

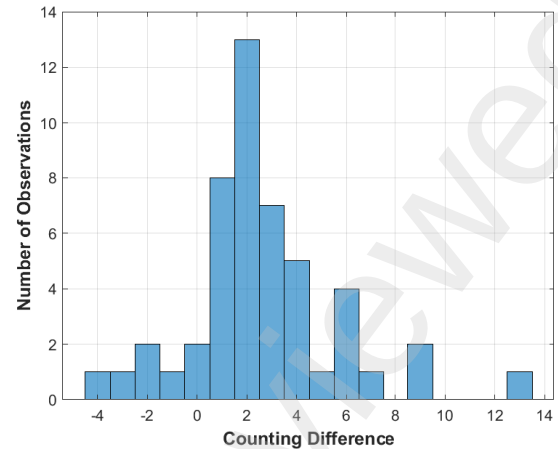


**Figure 22:** The results of the Linear Regression Analysis between the algorithmic counting versus the manual counting.

tively. It is worth mentioning that the apple counting method is highly similar to the manual counting results ( $R^2=0.95$ ). Meanwhile, the AppleYOLO maintains a high speed performance of 100.3FPS, ensuring stable and real-time counting capability. The results of these experiments evidence the great potential of the AppleYOLO to improve the efficiency for the Precision Agriculture, especially in meeting the requirements of high accuracy and timeliness of orchard yield estimation. Through the continuous optimisation of the AppleYOLO model size and generalization ability, we expect to provide more reliable and efficient technical support for the Precision Agriculture. We also expect to achieve high-precision management and yield estimation of diversified crops, expanding the application scope of the model.

## Funding

This work was supported in part by the National Natural Science Foundation of China under Grant 62072477, the



**Figure 23:** The difference between algorithmic counting and manual Counting.

Hunan Provincial Natural Science Foundation of China under Grant 2024JJ5648, the Scientific Research Fund of Hunan Provincial Education Department under Grant 18B197, the National Key R&D Program of China under Grant 2023YFD220170300, the Hunan Key Laboratory of Intelligent Logistics Technology 2019TP1015.

## CRedit authorship contribution statement

**Shiting Tan:** Writing – original draft preparation, Data acquisition, Visualization, Validation. **Zhufang Kuang:** Methodology, Writing – review & editing. **Boyu Jin:** Data acquisition, Formal analysis.

## References

- Azad, R., Niggemeier, L., Huttemann, M., Kazerouni, A., Aghdam, E.K., Velichko, Y., Bagci, U., Merhof, D., 2023. Beyond Self-Attention: Deformable Large Kernel Attention for Medical Image Segmentation *arXiv:2309.00121*.
- Bai, T., Zhang, N., Mercatoris, B., Chen, Y., 2019. Improving Jujube Fruit Tree Yield Estimation at the Field Scale by Assimilating a Single Landsat Remotely-Sensed LAI into the WOFOST Model. *Remote Sensing* 11. doi:10.3390/rs11091119.
- Cao, Y., Chen, J., Zhang, Z., 2023. A sheep dynamic counting scheme based on the fusion between an improved-sparrow-search YOLOv5x-ECA model and few-shot deepsort algorithm. *Computers and Electronics in Agriculture* 206, 107696. URL: <https://doi.org/10.1016/j.compag.2023.107696>.
- Chen, J., Kao, S.h., He, H., Zhuo, W., Wen, S., Lee, C.H., Chan, S.H.G., 2023. Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks, in: 2023

- IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12021–12031. doi:10.1109/CVPR52729.2023.01157.
- Chen, Z., Ho, P.H., 2021. A generic shift-norm-activation approach for deep learning. *Pattern Recognition* 109, 107609. URL: <https://doi.org/10.1016/j.patcog.2020.107609>.
- Cui, J., Zheng, H., Zeng, Z., Yang, Y., Ma, R., Tian, Y., Tan, J., Feng, X., Qi, L., 2023. Real-time missing seedling counting in paddy fields based on lightweight network and tracking-by-detection algorithm. *Computers and Electronics in Agriculture* 212, 108045. URL: <https://doi.org/10.1016/j.compag.2023.108045>.
- Ding, X., Zhang, X., Han, J., Ding, G., 2021. Diverse Branch Block: Building a Convolution as an Inception-like Unit, in: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10881–10890. doi:10.1109/CVPR46437.2021.01074.
- Du, X., Cheng, H., Ma, Z., Lu, W., Wang, M., Meng, Z., Jiang, C., Hong, F., 2023. DSW-YOLO: A detection method for ground-planted strawberry fruits under different occlusion levels. *Computers and Electronics in Agriculture* 214, 108304. URL: <https://doi.org/10.1016/j.compag.2023.108304>.
- Feng, K., Lun, L., Wang, X., Cui, X., 2023. LRTransDet: A Real-Time SAR Ship-Detection Network with Lightweight ViT and Multi-Scale Feature Fusion. *Remote Sensing* 15. doi:10.3390/rs15225309.
- Fu, L., F.Y.W.J.e.a., 2021. Fast and accurate detection of kiwifruit in orchard using improved YOLOv3-tiny model. *Precision Agric* 22, 754–776. URL: <https://doi.org/10.1007/s11119-020-09754-y>.
- Gao, C., Guo, W., Yang, C., Gong, Z., Yue, J., Fu, Y., Feng, H., 2024. A fast and lightweight detection model for wheat fusarium head blight spikes in natural environments. *Computers and Electronics in Agriculture* 216, 108484. URL: <https://doi.org/10.1016/j.compag.2023.108484>.
- Gao, F., Fang, W., Sun, X., Wu, Z., Zhao, G., Li, G., Li, R., Fu, L., Zhang, Q., 2022. A novel apple fruit detection and counting methodology based on deep learning and trunk tracking in modern orchard. *Computers and Electronics in Agriculture* 197, 107000. URL: <https://doi.org/10.1016/j.compag.2022.107000>.
- Guo, M.H., Liu, Z.N., Mu, T.J., Hu, S.M., 2023. Beyond Self-Attention: External Attention Using Two Linear Layers for Visual Tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 5436–5447. doi:10.1109/TPAMI.2022.3211006.
- Gómez-Lagos, J.E., González-Araya, M.C., Ortega Blu, R., Acosta Espejo, L.G., 2023. A new method based on machine learning to forecast fruit yield using spectrometric data: analysis in a fruit supply chain context. *Precision Agriculture* 24, 326–352. URL: <https://doi.org/10.1007/s11119-022-09947-7>.
- He, L., Fang, W., Zhao, G., Wu, Z., Fu, L., Li, R., Majeed, Y., Dhupia, J., 2022. Fruit yield prediction and estimation in orchards: A state-of-the-art comprehensive review for both direct and indirect methods. *Computers and Electronics in Agriculture* 195, 106812. URL: <https://www.sciencedirect.com/science/article/pii/S0168169922001296>.
- Huang, H., Huang, J., Wu, Y., Zhuo, W., Song, J., Li, X., Li, L., Su, W., Ma, H., Liang, S., 2023. The Improved Winter Wheat Yield Estimation by Assimilating GLASS LAI Into a Crop Growth Model With the Proposed Bayesian Posterior-Based Ensemble Kalman Filter. *IEEE Transactions on Geoscience and Remote Sensing* 61, 1–18. doi:10.1109/TGRS.2023.3259742.
- Lau, K.W., Po, L.M., Rehman, Y.A.U., 2024. Large Separable Kernel Attention: Rethinking the Large Kernel Attention design in CNN. *Expert Systems with Applications* 236, 121352. URL: <https://doi.org/10.1016/j.eswa.2023.121352>.
- Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., Li, Y., Zhang, B., Liang, Y., Zhou, L., Xu, X., Chu, X., Wei, X., Wei, X., 2022a. YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications *arXiv:2209.02976*.
- Li, C., Yao, A., 2023. KernelWarehouse: Towards Parameter-Efficient Dynamic Convolution *arXiv:2308.08361*.
- Li, C., Zhou, A., Yao, A., 2022b. Omni-Dimensional Dynamic Convolution, in: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=DmpCfq6Mg39>.
- Li, Y., Ma, R., Zhang, R., Cheng, Y., Dong, C., 2023. A Tea Buds Counting Method Based on YOLOv5 and Kalman Filter Tracking Algorithm. *Plant Phenomics* 5, 0030. doi:10.34133/plantphenomics.0030.
- Li, Z., Jiang, J., Chen, X., Zhang, M., Wang, Y., Li, Q., Qi, H., Liu, M., Laganière, R., 2024a. Scale-pyramid dynamic atrous convolution for pixel-level labeling. *Expert Systems with Applications* 241, 122695. URL: <https://doi.org/10.1016/j.eswa.2023.122695>.
- Li, Z., Zhu, Y., Sui, S., Zhao, Y., Liu, P., Li, X., 2024b. Real-time detection and counting of wheat ears based on improved YOLOv7. *Computers and Electronics in Agriculture* 218, 108670. URL: <https://doi.org/10.1016/j.compag.2024.108670>.

- Liu, J.J., Hou, Q., Cheng, M.M., Wang, C., Feng, J., 2020. Improving Convolutional Networks With Self-Calibrated Convolutions, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 10093–10102. doi:10.1109/CVPR42600.2020.01011.
- Liu, X., Peng, H., Zheng, N., Yang, Y., Hu, H., Yuan, Y., 2023. EfficientViT: Memory Efficient Vision Transformer with Cascaded Group Attention, in: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 14420–14430. doi:10.1109/CVPR52729.2023.01386.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B., 2021. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows abs/2103.14030, 9992–10002. arXiv:2103.14030.
- Lv, W., Xu, S., Zhao, Y., Wang, G., Wei, J., Cui, C., Du, Y., Dang, Q., Liu, Y., 2023. DETRs Beat YOLOs on Real-time Object Detection arXiv:2304.08069.
- Maggiolino, G., Ahmad, A., Cao, J., Kitani, K., 2023. Deep OC-Sort: Multi-Pedestrian Tracking by Adaptive Re-Identification, in: 2023 IEEE International Conference on Image Processing (ICIP), pp. 3025–3029. doi:10.1109/ICIP49359.2023.10222576.
- Mirhaji, H., Soleymani, M., Asakereh, A., Abdanan Mehdizadeh, S., 2021. Fruit detection and load estimation of an orange orchard using the YOLO models through simple approaches in different imaging and illumination conditions. Computers and Electronics in Agriculture 191, 106533. URL: <https://doi.org/10.1016/j.compag.2021.106533>.
- Oliveira, L.S.d., Castoldi, R., Martins, G.D., Medeiros, M.H., 2023. Estimation of Strawberry Crop Productivity by Machine Learning Algorithms Using Data from Multispectral Images. Agronomy 13. doi:10.3390/agronomy13051229.
- Redmon, J., Farhadi, A., 2018. YOLOv3: An Incremental Improvement. Computing Research Repository abs/1804.02767.
- Ren, W., Wang, X., Tian, J., Tang, Y., Chan, A.B., 2021. Tracking-by-Counting: Using Network Flows on Crowd Density Maps for Tracking Multiple Targets. IEEE Transactions on Image Processing 30, 1439–1452. doi:10.1109/TIP.2020.3044219.
- Rong, J., Zhou, H., Zhang, F., Yuan, T., Wang, P., 2023. Tomato cluster detection and counting using improved YOLOv5 based on RGB-D fusion. Computers and Electronics in Agriculture 207, 107741. URL: <https://doi.org/10.1016/j.compag.2023.107741>.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C., 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4510–4520. doi:10.1109/CVPR.2018.00474.
- Shen, L., Su, J., He, R., Song, L., Huang, R., Fang, Y., Song, Y., Su, B., 2023. Real-time tracking and counting of grape clusters in the field based on channel pruning with YOLOv5s. Computers and Electronics in Agriculture 206, 107662. URL: <https://www.sciencedirect.com/science/article/pii/S0168169923000509>.
- Wang, C.Y., Bochkovskiy, A., Liao, H.Y.M., 2023a. YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7464–7475.
- Wang, C.Y., Liao, H.Y.M., 2024. YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information arXiv:2402.13616.
- Wang, W., Dai, J., Chen, Z., Huang, Z., Li, Z., Zhu, X., Hu, X., Lu, T., Lu, L., Li, H., Wang, X., Qiao, Y., 2023b. InternImage: Exploring Large-Scale Vision Foundation Models with Deformable Convolutions, in: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 14408–14419. doi:10.1109/CVPR52729.2023.01385.
- Wulfsohn, D., Aravena Zamora, F., Potin Téllez, C., Zamora Lagos, I., García-Fiñana, M., 2012. Multi-level systematic sampling to estimate total fruit number for yield forecasts. Precision Agriculture 13, 256–275. doi:10.1007/s11119-011-9245-2.
- Xia, X., Chai, X., Zhang, N., Zhang, Z., Sun, Q., Sun, T., 2022. Culling Double Counting in Sequence Images for Fruit Yield Estimation. Agronomy 12. doi:10.3390/agronomy12020440.
- Xiong, Z., Wang, L., Zhao, Y., Lan, Y., 2023. Precision Detection of Dense Litchi Fruit in UAV Images Based on Improved YOLOv5 Model. Remote Sensing 15. URL: <https://www.mdpi.com/2072-4292/15/16/4017>.
- Yang, J., Li, C., Dai, X., Gao, J., 2022. Focal Modulation Networks. Advances in Neural Information Processing Systems (NeurIPS) doi:10.48550/arXiv.2203.11926.
- Ye, T., Zhang, J., Zhao, Z., Zhou, F., 2022. Foreign Body Detection in Rail Transit Based on a Multi-Mode Feature-Enhanced Convolutional Neural Network. IEEE Transactions on Intelligent Transportation Systems 23, 18051–18063. doi:10.1109/TITS.2022.3154751.
- Zhang, W., Wang, J., Liu, Y., Chen, K., Li, H., Duan, Y., Wu, W., Shi, Y., Guo, W., 2022. Deep-learning-based

in-field citrus fruit detection and tracking. Horticulture Research 9, uhac003. doi:10.1093/hr/uhac003.

Zhang, Y., Zhou, G., Chen, A., He, M., Li, J., Hu, Y., 2023. A precise apple leaf diseases detection using BCTNet under unconstrained environments. Computers and Electronics in Agriculture 212, 108132. URL: <https://doi.org/10.1016/j.compag.2023.108132>.

Üzen, H., Türkoğlu, M., Yanikoglu, B., Hanbay, D., 2022. Swin-MFNet: Swin transformer based multi-feature integration network for detection of pixel-level surface defects. Expert Systems with Applications 209, 118269. URL: <https://doi.org/10.1016/j.eswa.2022.118269>.